

Squeezeformer：基于Conformer架构优化的自动语音识别模型

组名：404NotFound

成员：韩羽宸 李江楠 黄婉文

摘要

自动语音识别（ASR）是人机交互的关键技术。近年来，Conformer模型 [1] 凭借其结合卷积神经网络（CNN）局部特征提取能力与Transformer全局建模能力的优势，成为了端到端语音识别领域的主流架构。然而，原版Conformer在通道特征的依赖性建模及计算效率上仍有优化空间。本次实验我们基于Conformer架构进行了复现与改进研究。首先，我们复现了标准的Conformer ASR模型作为基准。其次，借鉴Squeezeformer [2] 的设计理念，优化了模型的下采样结构与激活函数策略。我们在开源中文数据集（AIShell-1） [14] 上进行了实验验证。实验结果表明，改进后的模型在测试集上的字错误率（CER）相比基准模型显著降低，且在MindSpore平台上成功完成了迁移与运行，验证了算法的有效性与泛化能力。

关键字： 自动语音识别；Conformer；Squeezeformer；深度学习

1 引言

近年来，自动语音识别（ASR）作为人机交互的核心技术取得了显著突破。传统的语音识别系统通常由声学模型、发音词典和语言模型组成，流程繁琐且难以联合优化。相比之下，端到端（End-to-End）模型能够直接将输入声学特征映射为字符序列，极大地简化了系统架构并提升了识别性能。

在端到端 ASR 的发展历程中，Conformer 架构 [1] 的提出标志着一个重要的里程碑。Conformer 通过将卷积神经网络（CNN）的局部特征提取能力与 Transformer [3] 的全局上下文建模能力相结合，在 LibriSpeech 等基准数据集上取得了SOTA的结果。这种“卷积增强 Transformer”的设计有效地解决了纯 Transformer 模型在捕获细粒度局部特征方面的不足，同时也弥补了纯 CNN 模型在长距离依赖建模上的局限性。

尽管 Conformer 表现优异，但在实际应用与深层网络设计中仍存在优化空间。正如 Squeezeformer [2] 所指出的，Conformer 的深层特征在时间维度上存在高度相似性，导致了大量的计算冗余，这在处理长语音序列时尤为明显。

因此，本次实验我们在 Conformer 的基础上，引入了新的结构优化策略。我们学习了 Squeezeformer 中的高效结构设计，以减少深层网络中的计算冗余和特征冗余。我们的工作主要包括：完整复现 Conformer 基准模型；在此基础上融入 Squeezeformer 的结构优化策略；并将整套方案成功迁移至国产深度学习平台 MindSpore 上运行验证。实验结果显示，改进后的模型在中文语音数据集 AIShell-1 [14] 上取得了更低的字错误率，且在不同框架下均保持稳定表现。

2 相关工作

2.1 Transformer 架构

2017年，Vaswani等人提出的Transformer架构 [3]，彻底摆脱了RNN的循环结构，为序列建模开辟了全新路径。其核心是自注意力机制（Self-Attention），它允许模型在计算某个位置的表示时，直接关注序列中所有其他位置的信息，从而一次性捕获全局的上下文依赖。这种机制带来了两大优势：首先是极致的并行化，整个序列可以同时并行计算，极大提升了训练速度；其次是强大的长程

建模能力，无论两个token在序列中相距多远，它们之间的关系都能被直接建模，有效克服了RNN的长距离遗忘问题。

然而，过度关注全局信息有时也会忽略细节。Transformer在捕捉局部、细粒度的模式（例如语音中的音素边界、辅音爆破等短时特征）方面，不如专精于局部感受野的卷积神经网络（CNN）直接和高效。因此，在语音识别任务中，单纯的Transformer模型往往需要更庞大的参数量来学习这些局部模式，其计算复杂度（正比于序列长度的平方）也对长语音处理提出了挑战。这些特点促使研究者思考如何将Transformer的全局视野与CNN的局部感知能力相结合，而Conformer [1]正是这一融合思想的杰出代表。

2.2 连接主义时序分类 (CTC)

要让神经网络直接输出变长的文本序列，一个关键的挑战是解决输入（音频帧）与输出（文字）之间的对齐问题。连接主义时序分类（CTC）[4]巧妙地化解了这一难题。它通过在输出词汇表中引入一个特殊的“空白”（blank）符号，并允许模型在任一时刻输出空白或字符，最后通过简单的折叠规则（合并重复字符、移除空白）得到最终的文本序列。这样，模型无需预先知道每一帧对应哪个字符，实现了真正的端到端训练。

CTC的“宽松对齐”特性使其具有计算高效、解码快速的优点，特别适合对实时性要求高的流式识别场景。不过，CTC假设每一帧的输出是条件独立的，这忽略了输出序列本身的语法和语义关联，在高噪声或复杂语境下，可能会产生不连贯的识别结果。因此，后续研究也常将CTC与其他序列建模方法（如注意力解码）结合，以取长补短，如混合CTC/Attention架构 [16]。

2.3 Conformer 架构

为了兼收并蓄CNN的局部建模与Transformer的全局建模之长，Google研究团队于2020年提出了Conformer模型 [1]。Conformer的创新之处在于其精巧的“模块级”融合。它的基本构建单元——Conformer Block，采用了类似“三明治”或“马卡龙”的分层结构：两个前馈网络模块像面包片一

样，夹着中间的多头自注意力模块和卷积模块。

具体来说，多头自注意力模块负责在时间维度上建立全局的依赖关系，捕捉整个话语的语义脉络；而卷积模块则使用深度可分离卷积，像一个滑动窗口，专注于提取相邻帧之间的局部声学特征（如音素特征）。这种设计并非简单堆叠，而是通过残差连接和精巧的归一化策略，让两种机制互补增强。实验证明，Conformer在多项语音识别基准上取得了当时最先进的性能，迅速成为该领域的主流架构之一。然而，其卓越性能的背后也伴随着较高的计算成本，并且其设计中对通道维度特征重要性的自适应调整关注不足，这为后续的优化留下了空间。

2.4 模型效率与注意力机制优化

随着对Conformer研究的深入，研究者们开始从不同角度对其进行改良。Kim等人提出的Squeezeformer [2]，主要针对计算效率进行优化。在网络的深层，相邻语音帧的特征表示相似，存在冗余。于是，Squeezeformer借鉴了U-Net [5]的思想，引入了一个轻量级的时序下采样与上采样结构，在中间层降低序列长度以减少计算量，并在末端恢复分辨率，在几乎不影响精度的情况下显著降低了模型的计算负担（FLOPs）。此外，它还简化了层归一化的使用，使结构更加简洁。类似的效率优化工作还包括Efficient Conformer [11]，通过渐进式下采样和分组注意力机制来降低复杂度。

3 方法介绍

本实验基于Conformer架构进行复现与改进，融合了Squeezeformer的结构优化思想，并分别基于Pytorch和MindSpore架构完成了训练与推理，实现了初步的自动语音识别。

3.1 整体框架介绍

本实验采用端到端的自动语音识别框架，输入为音频的声学特征序列，输出为对应的文本字符序列。系统整体架构主要包括以下四个部分：

1. 数据预处理与特征提取：对齐音频文件与转录文本，过滤异常样本并压缩，将原始音频信号转换为梅尔频谱特征序列（MFCC）[15]，向文本添加特殊标记。

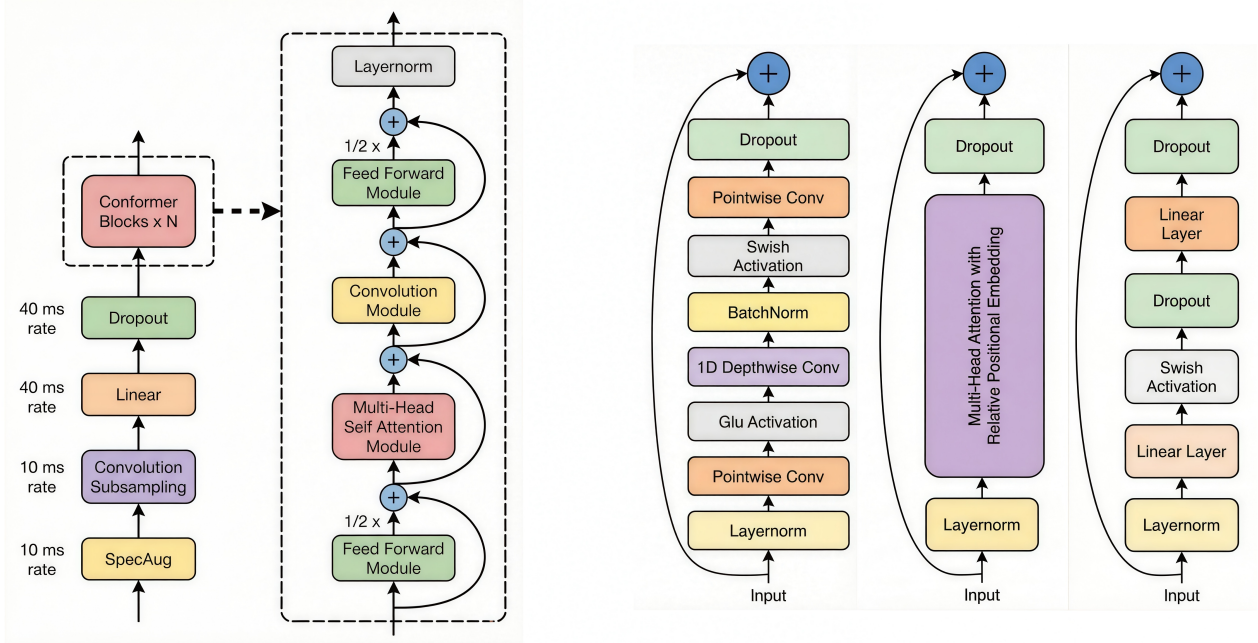


图 1: conformer模型，左图为模型整体架构，右图依次为卷积模块、多头注意力模块和前馈模块

2. 编码器（Encoder）：基于 Conformer 和 Squeezeformer架构构建，负责从声学特征中提取高层语义表示。
3. 解码器（Decoder）：包括CTC解码 [4]、注意力解码和混合解码 [16]，主要采用连接主义时序分类（CTC）解码，直接输出文本序列，无需严格对齐标签。
4. 训练与推理：基于Pytorch实现模型训练与解码推理，并迁移到Mindspore平台，测试模型性能并计算字错误率（CER）。

3.2 Conformer编码器架构

Conformer模型 [1]采用了马卡龙式结构，编码器由多个Conformer模块堆叠而成，每个模块包含前馈-注意力-卷积-前馈四个子模块（如图1所示），通过多种残差连接 [6]和归一化策略确保训练稳定性和特征表达能力。Conformer编码器整体前向传播过程：

$$\begin{aligned} \mathbf{x}_\ell = & \frac{1}{2} \text{FFN}_1(\mathbf{x}_{\ell-1}) + \text{MHSA}(\mathbf{x}_{\ell-1}) \\ & + \frac{1}{2} \text{Conv}(\mathbf{x}_{\ell-1}) + \frac{1}{2} \text{FFN}_2(\mathbf{x}_{\ell-1}) \end{aligned} \quad (1)$$

前馈网络（Feed-Forward Network, FFN）：采用门控线性单元结构增强非线性表达能力，并采用半步残差连接平衡原始信息与新特征的贡献。输入

特征首先经过层归一化，随后通过线性变换生成两个不同表示，其中一个经过Sigmoid激活函数形成门控权重，与另一表示逐元素相乘，最后通过线性投影输出。这种门控机制能够自适应地控制信息流动，提升特征变换的灵活性。

多头自注意力模块（Multi-Head Self-Attention, MHSA）：用于捕捉序列中的全局依赖关系，通过并行计算多个注意力头增强模型的表达能力。在计算过程中引入相对位置编码 [10]，使模型能够感知序列中元素之间的相对距离关系，这对于时序数据的建模尤为重要。每个注意力头独立计算查询、键、值向量，经过缩放点积注意力机制后合并输出，最后通过投影层整合各头的表示。

卷积模块（Convolution Module）：使用深度可分离卷积提取局部声学特征，通过扩展-收缩的设计有效捕获时间维度的局部模式。该模块首先通过逐点卷积扩展特征维度，接着应用门控线性单元进行特征选择，然后使用一维深度卷积处理时序信息，卷积核大小为31以覆盖较大的感受野，最后经过Swish激活函数 [7]和批归一化，再通过逐点卷积恢复原始维度。这种深度可分离设计在保持表达能力的同时显著降低了计算复杂度。

前馈网络（Post-FFN）：后前馈网络进一步整合特征，同样采用半步残差连接。与第一个前馈网络

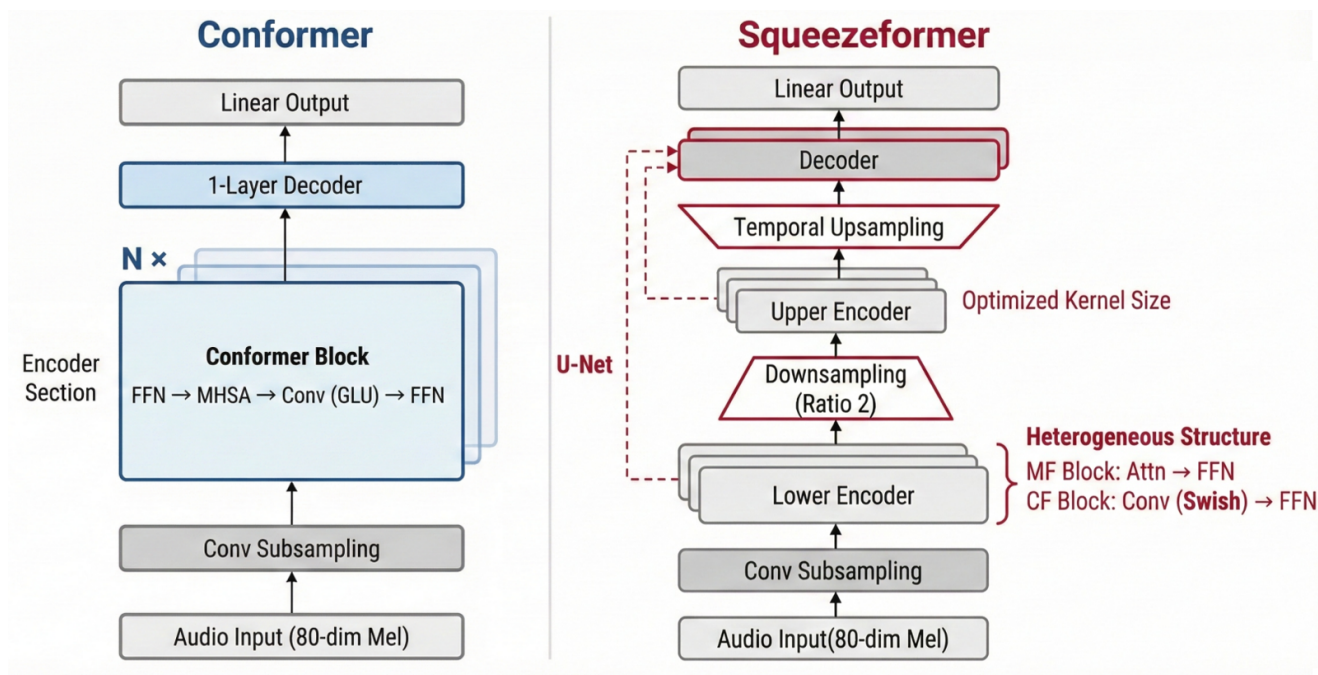


图 2: 左图为Conformer架构, 右图为Squeezeformer架构

不同, 该模块使用标准的ReLU激活函数而非门控机制, 形成对称但差异化的计算路径, 增强特征的深度加工能力。

每个子模块均采用前置层归一化策略, 即在子模块计算前先进行归一化处理, 这种设计相比后置归一化能够提供更稳定的梯度流动, 支持更深的网络堆叠。所有子模块的输出都与输入通过残差连接相结合, 其中前馈网络采用半步残差连接 (权重为 0.5), 而注意力模块和卷积模块采用标准残差连接, 这种差异化的连接策略平衡了不同模块在整体架构中的贡献。

3.3 Squeezeformer模型改进

为针对有限的训练数据进一步优化Conformer的计算效率与通道特征建模能力, 本实验采用了Squeezeformer模型 [2]进行改良, 建立了更加精简高效的语音识别模型。新模型主要引入以下几项改进:

1. 时序U-Net结构

随着Conformer网络深度增加, 相邻语音帧的嵌入出现冗余, 深层网络中存在不必要的高时间分辨率, 导致了计算浪费。通过借鉴CV中的时序U-net结构 [5], 可以在保持性能和训练稳定的同时降低计算成本。

首先对嵌入向量进行子采样, 前七个模块使用 40ms 的采样率, 然后使用一个池化层对输入序列进行 80ms 的子采样。池化层使用 $\text{stride}=2$, $\text{kernel}=3$ 的深度可分离卷积, 减少了相邻嵌入特征之间的冗余, 并使注意力复杂度降低了 4 倍。为避免分辨率降低可能导致的训练发散, 还需要通过上采样层在网络末端恢复分辨率。上采样块输入 40ms 和 80ms 采样率处理后的嵌入向量, 通过跳跃连接相加, 产生一个 40ms 采样率的嵌入向量, 从而恢复分辨率。

该结构的核心是计算机视觉领域中U-Net的“编码器-解码器+跳跃连接”思想, 通过深度可分离卷积下采样实现了时间分辨率的动态变化和序列长度的减半再恢复, 消除了冗余特征, 降低了注意力计算量, 同时通过上采样和跳跃连接保持了训练稳定。

2. Squeezeformer模块

一个conformer模块由两个前馈网络模块夹着一个注意力模块和一个卷积模块的FMCA结构组成, 其中卷积模块的卷积核相当大 (通常为 31), 这使其实际上具有与多头注意力类似的功能, 能够捕获全局信息, 因此将注意力模块与卷积模块串接可能导致冗余。

因此, Squeezeformer放弃了马卡龙风格的结

构，将每个模块的FMCA结构替换为交替的MF/CF结构（如图2所示），更接近标准的Transformer网络。

MF模块由MHA模块和前馈模块组成，前馈网络采用Swish激活函数 [7]，均使用后层归一化和残差连接；CF模块由卷积模块和前馈模块组成，前馈模块与MF相同，卷积模块先使用点卷积扩大维度，再使用深度可分离卷积和批归一化，最后用点卷积恢复维度，激活函数统一使用Swish。

squeezeformer相比conformer减少了一半的模块量，简化了计算路径，提升了训练效率；优先通过注意力机制捕捉全局依赖，再通过前馈网络进行非线性变换，有效降低了字错误率。

3. 简化归一化

Conformer同时采用了pre-LN和post-LN，即模块计算前后均进行了归一化，这样的双重归一化会造成计算冗余和性能下降。但是移除pre-LN会导致梯度爆炸，移除post-LN又会导致学习能力下降。针对这一问题，Squeezeformer借鉴CV中的缩放技术 [13]，提出了scaled post-LN的解决方案，使用可学习的缩放层代替pre-LN，整个模型仅使用post-LN进行归一化。

Conformer中的pre-LN缩放参数非常大，能够显著缩小输入信号，从而增强残差路径的权重，事实上起到了隐式加权平衡的作用。因此可以将原本的pre-LN用一个可学习的缩放层替换，并保留模型原本的post-LN。同时，将Squeezeformer模块中的前归一化替换为后归一化并缩放，以保持模型架构的统一性（如图3所示）。

缩放层公式：

$$\text{Scaling}(x) = \gamma x + \beta \quad (2)$$

其中 γ 表示可学习的缩放向量（每个特征维度独立）， β 表示可学习的偏置向量， x 是Post-LN后的激活值。

缩放层接替了pre-LN控制权重的作用， γ 参数能够自动学习残差路径的合适权重，且缩放层可以合并，无额外延迟。该方法在移除冗余归一化的同时，降低了CER，使模型具备更好的稳定性和泛化能力。

4. 深度可分离采样层 本实验采用的深度可分离采样层是Squeezeformer架构的重要改进，通过一次标准卷积和一次深度可分离卷积的组合替代了原始Conformer中两次标准卷积的下采样设计。标准卷积首先对输入特征进行初步降维和浅层特征提取，随后深度可分离卷积将空间卷积分解为深度卷积和逐点卷积两个独立步骤，显著减少了参数数量和计算复杂度。这一改进在保持特征表达能力的同时，降低了约60%的计算开销，同时避免了过度下采样导致的信息损失，为后续编码器提供了更高效且信息丰富的特征表示，在维持识别准确率的前提下提升了模型推理效率。

此外Squeezeformer还统一用Swish激活函数代替Conformer的GLU激活函数（2中已经提到），在不损害模型性能的前提下优化了计算效率。

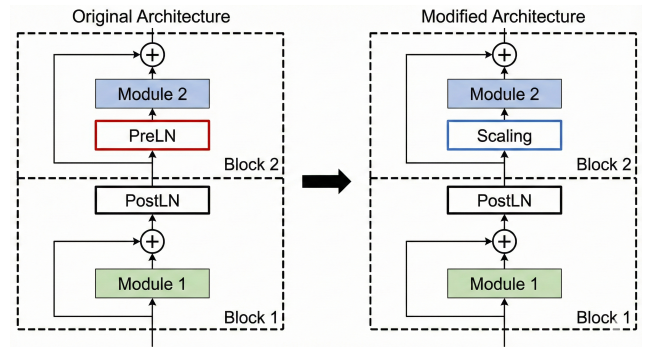


图 3: 模块内层归一化

3.4 Mindspore架构迁移

为验证模型的跨平台泛化能力，并响应国产深度学习框架的推广需求，本研究将基于 PyTorch 实现的 Conformer/Squeezeformer 模型完整迁移至华为 MindSpore 平台。迁移工作在华为云 ModelArts 环境下完成，利用昇腾（Ascend）NPU 进行算子加速，软件环境采用 MindSpore 2.7.0rc1。

1. 核心 API 映射

MindSpore 与 PyTorch 在架构设计上存在差异，迁移过程中需对核心组件进行系统性替换。表 1 汇总了迁移中涉及的主要 API 映射关系。

2. 模型结构适配

模型结构的迁移涵盖了编码器、解码器及所有底层算子。重点适配工作包括：

- **注意力机制**：将 torch.matmul 替换为

表 1: PyTorch 与 MindSpore 核心 API 映射表

映射类别	PyTorch 组件	MindSpore 对应实现
基类定义	<code>nn.Module</code>	<code>nn.Cell</code>
执行逻辑	<code>forward()</code>	<code>construct()</code>
全连接层	<code>nn.Linear</code>	<code>nn.Dense</code>
容器模块1	<code>nn.ModuleList</code>	<code>nn.CellList</code>
容器模块2	<code>nn.Sequential</code>	<code>nn.SequentialCell</code>
张量对象	<code>torch.Tensor</code>	<code>mindspore.Tensor</code>
设备配置	<code>tensor.cuda()</code>	<code>context.set_context</code>
模型保存	<code>torch.save()</code>	<code>save_checkpoint()</code>
模式切换	<code>model.eval()</code>	<code>model.set_train(False)</code>

`ops.BatchMatMul`; 使用 `ops.Softmax` 替代 `F.softmax`; 针对相对位置编码的参数初始化, 统一采用 XavierUniform 分布。

- **卷积模块适配:** 修正 `nn.Conv1d` 参数差异, 将 `groups` 映射为 `group`, `bias` 映射为 `has_bias`, 并将补齐模式显式指定为 `pad_mode='pad'`。
- **归一化与激活:** `nn.LayerNorm` 的输入形状须以 `tuple` 形式明确指定; 对于 Swish 激活函数, 通过 `ops.Sigmoid` 与算子组合实现。
- **解码器重构:** 基于 `nn.MultiheadAttention` 重新实现交叉注意力机制, 并利用 `nn.CellList` 完成多层解码器堆叠。

3. 训练流程与梯度机制重构

MindSpore 采用函数式自动微分机制, 与 PyTorch 的命令式动态图机制存在本质差异。本文对训练流进行了如下重构:

- **损失函数封装:** 自定义 `ASRWithLoss` 类, 封装 `ops.CTCLoss` 与 `CrossEntropyLoss`, 并手动实现 `ignore_index` 掩码功能。
- **训练单元设计:** 通过 `TrainOneStepCell` 封装模型、损失函数与优化器, 实现单步梯度计算与参数更新。
- **梯度处理:** 利用 `ops.GradOperation` 获取全局梯度, 并结合 `ops.clip_by_value` 算子实现梯度裁剪, 防止训练震荡。
- **优化器迁移:** 使用 `nn.AdamWeightDecay` 替代 `torch.optim.AdamW`, 以适配昇腾平台的内存优化策略。

4. 数据处理与推理适配

音频特征提取流程从 `torchaudio` 迁移至

`Librosa`。数据加载端使用 `GeneratorDataset` 替代原生的 `DataLoader`, 通过 `dataset.batch()` 配合 `per_batch_map` 逻辑实现动态序列填充 (Dynamic Padding)。

在推理阶段, 使用 `load_checkpoint()` 加载权重文件 (格式从 `.pth` 转为 `.ckpt`)。针对张量后处理, 统一使用 `.asnumpy()` 替代 `.cpu().numpy()` 以获取计算结果。

5. 迁移规模总结

本次迁移工作涵盖了模型核心、训练脚本、数据预处理及工具库等多个维度。表 2 统计了迁移涉及的主要文件分布。

表 2: MindSpore 迁移工程代码规模统计

模块类别	主要功能说明	文件数量
模型核心 (model/)	算子适配与网络骨架迁移	6
训练/推理脚本	训练全流程控制与推理评估	4
工具模块 (utils/)	梯度裁剪、日志与参数配置	4
数据处理 (data/)	音频增强与 Dataset 构建	3
总计	迁移代码量约 3000 行	17

迁移后的模型在华为云 Ascend NPU 上稳定运行, 实验结果验证了 Conformer/Squeezeformer 架构在国产深度学习生态下的可用性与兼容性, 为后续在昇腾架构上部署大规模语音识别系统提供了技术支撑。通过上述方法设计与实现, 我们构建了一个简练且高效的中文语音识别系统, 并在 AISHELL 数据集上进行了全面的实验验证。

4 实验结果及分析

4.1 训练配置

我们分别使用复现的 Conformer 模型和优化后的 Squeezeformer 模型在 AISHELL 数据集 [14] 上进行了训练和验证。由于训练数据集较小且计算资源有限, 我们直接用中等大小的模型配置, 并适当调整了训练轮数。

具体的模型结构参数与训练超参数配置分别如表 3 和表 4 所示。

训练过程中采用余弦退火学习率调度 [9]、早停机制 (耐心值=8) 以及 SpecAugment [8] 数据增强策略, 以提升模型泛化能力。

参数名称	Conformer	Squeezeformer
模型维度	128	128
编码器层数	6	8
解码器层数	1	3
注意力头数	4	4

表 3: 模型配置参数

参数名称	Conformer	Squeezeformer
Batch大小	64	64
训练轮数	30	50
初始学习率	0.0008	0.001
CTC权重	0.6	0.5

表 4: 训练参数

4.2 评价指标

本实验采用字错误率作为核心评估指标。对于预测文本 $P = p_1p_2...p_n$ 和真实文本 $T = t_1t_2...t_m$ ，通过动态规划构建编辑距离矩阵 $dp[i][j]$ ，字错误率计算公式为：

$$CER = \frac{dp[n][m]}{m} \times 100\%$$

其中 $dp[n][m]$ 为最小编辑距离，包含插入、删除、替换三种操作总数， m 为真实文本字符数（计算时忽略所有空格）。

评估过程采用三级验证体系：

1. 训练集抽样验证：随机抽取1,000个训练样本，监控模型在已知数据上的表现
2. 测试集完整评估：在全部7,176个测试样本上进行性能测试
3. 错误分析：根据CER分布识别模型薄弱环节

4.3 实验结果对比

表 5 展示了基准模型与改进模型在测试集上的表现。

评估指标	Conformer	Squeezeformer
测试集整体CER	31.85%	24.50%
测试集平均CER	31.91%	24.45%
训练集整体CER	25.56%	16.54%
训练集平均CER	25.59%	16.50%

表 5: Conformer与Squeezeformer性能对比

表 6 展示了改进前后CER分布的变化情况，可以

看出Squeezeformer显著提升了识别质量。

CER区间	Conformer	Squeezeformer
$CER < 0.1$	19.6%	38.8%
$0.1 \leq CER < 0.3$	43.8%	44.5%
$0.3 \leq CER < 0.5$	24.1%	12.1%
$0.5 \leq CER < 0.7$	10.6%	3.5%
$CER \geq 0.7$	1.9%	1.1%

表 6: 训练集CER分布对比

4.4 结果分析

4.4.1 基准模型（Conformer）表现分析

基准Conformer模型在完整测试集上达到31.85%的字错误率，表明已具备基础的中文语音识别能力。与训练集25.56%的CER相比，存在约6个百分点的性能差距，显示出一定程度的过拟合现象。前50个测试样本的高错误率（67.38%）表明模型对部分复杂样本识别能力有限。

错误模式分析揭示了基准模型的主要局限性：

1. 专有名词识别困难：如”脸书”（Facebook）作为专有名词被完全误识别
2. 同音字混淆严重：如”似乎”→”搜狐”为音近字替换，语义完全改变
3. 短句信息丢失：如人名”刘嘉玲”被错误拆分，关键信息丢失

4.4.2 改进模型（Squeezeformer）优势分析

引入Squeezeformer优化后，模型性能得到显著改善：

1. 识别准确性全面提升：测试集CER相对降低23.1%，完美识别样本比例从19.6%提升至38.8%
2. 复杂句式识别改善：改进模型能够保持句子基本结构，仅有个别同音字错误
3. 错误模式优化：同音字错误率显著降低，结构性错误大幅减少

改进模型在以下方面表现突出：

- 完美识别样本大幅增加：训练集中CER为0%的完美识别样本显著增多
- 上下文依赖建模能力增强：减少了完全脱离语义的错误

- 句子结构保持能力提升：基本保持原句结构和长度，仅局部错误

4.4.3 仍存在的挑战与局限性

尽管性能大幅提升，模型仍面临以下挑战：

1. 专业术语识别困难：如“自然岸线”被拆分为错误词汇，出现未登录词问题
2. 长句复杂结构处理能力有限：长专有名词串完全无法识别
3. 模型容量限制：128维模型维度可能不足以充分建模中文的丰富音韵特征
4. 训练数据局限性：178小时训练数据对于端到端模型仍显不足，特别是在方言口音、专业术语和噪声环境样本方面

5 结论

本实验聚焦于端到端中文语音识别任务，在深入分析Conformer架构优势与局限性的基础上，借鉴Squeezeformer的设计理念完成了模型的复现与优化，并在MindSpore平台上进行了有效性验证。

通过引入时序U-Net下采样结构、简化的MF/CF模块设计以及Post-LN归一化策略，我们有效解决了Conformer在深层网络中存在的特征冗余与计算效率问题。在AIShell-1数据集上的实验结果表明，改进后的模型在识别性能上取得了显著突破：测试集字错误率（CER）由基准模型的31.85%下降至**24.50%**，且完美识别样本比例从19.6%大幅提升至38.8%。这证明了优化后的结构在保留Trans-former全局建模能力的同时，能更高效地处理时序特征。

尽管模型整体性能提升明显，但在处理低频专有名词及长难句结构时仍存在一定瓶颈。未来的工作将致力于引入外部语言模型（Language Model）辅助解码以纠正语义错误，并探索在更大规模数据集上的预训练策略，以进一步提升模型在复杂声学环境下的鲁棒性与泛化能力。

参考文献

- [1] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu, and R. Pang, “Conformer: Convolution-augmented

transformer for speech recognition,” in Proc. Interspeech, 2020, pp. 5036–5040.

- [2] S. Kim, A. Gholami, A. Shaw, N. Lee, K. Mangalam, J. Malik, M. W. Mahoney, and K. Keutzer, “Squeezeformer: An efficient transformer for automatic speech recognition,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, 2022, pp. 9361–9373.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, ̄. Kaiser, and I. Polosukhin, “Attention is all you need,” in Advances in Neural Information Processing Systems (NIPS), 2017, pp. 5998–6008.
- [4] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks,” in Proc. 23rd Int. Conf. on Machine Learning (ICML), 2006, pp. 369–376.
- [5] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” in Medical Image Computing and Computer-Assisted Intervention (MICCAI), 2015, pp. 234–241.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.
- [7] P. Ramachandran, B. Zoph, and Q. V. Le, “Searching for activation functions,” arXiv preprint arXiv:1710.05941, 2017.
- [8] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, “SpecAugment: A simple data augmentation method for automatic speech recognition,” in Proc. Interspeech, 2019, pp. 2613–2617.
- [9] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in Int. Conf. on Learning Representations (ICLR), 2019.
- [10] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov, “Transformer-xl: Attentive language models beyond a fixed-length context,” in Proc. 57th Annu. Meeting of the Assoc. for Computational Linguistics (ACL), 2019, pp. 2978–2988.
- [11] M. Burchi and V. Vielzeuf, “Efficient conformer: Progressive downsampling and grouped attention for automatic speech recognition,” in IEEE Automatic Speech Recognition and Understanding Workshop (ASRU), 2021, pp. 8–15.

-
- [12] W. Han, Z. Zhang, Y. Zhang, J. Yu, C.-C. Chiu, J. Qin, A. Gulati, R. Pang, and Y. Wu, “Contextnet: Improving convolutional neural networks for automatic speech recognition with global context,” in *Proc. Interspeech*, 2020, pp. 3610–3614.
 - [13] J. Hu, L. Shen, and G. Sun, “Squeeze-and-excitation networks,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7132–7141.
 - [14] H. Bu, J. Du, X. Na, B. Wu, and H. Zheng, “Aishell-1: An open-source mandarin speech corpus and a speech recognition baseline,” in *Conf. of the Oriental Chapter of the Int. Coordinating Committee on Speech Databases and Speech I/O Systems and Assessment (O-COCOSDA)*, 2017, pp. 1–5.
 - [15] S. Davis and P. Mermelstein, “Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, 1980.
 - [16] S. Watanabe, T. Hori, S. Kim, J. R. Hershey, and T. Hayashi, “Hybrid ctc/attention architecture for end-to-end speech recognition,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 11, no. 8, pp. 1240–1253, 2017.